

ELEMENTOS EN SECUENCIA

EJEMPLOS



RECORRIDOS

- Exhaustivos

Recorreremos la secuencia HASTA EL FINAL SIEMPRE, en todos los casos posibles.

- No exhaustivos

Pueden existir casos para los cuales NO SEA NECESARIO recorrer la secuencia por completo, es decir, sin necesidad de llegar al final podemos dar la respuesta al problema.

FIN DE SECUENCIA

- Conocemos la longitud de la secuencia antes de empezar a recorrerla

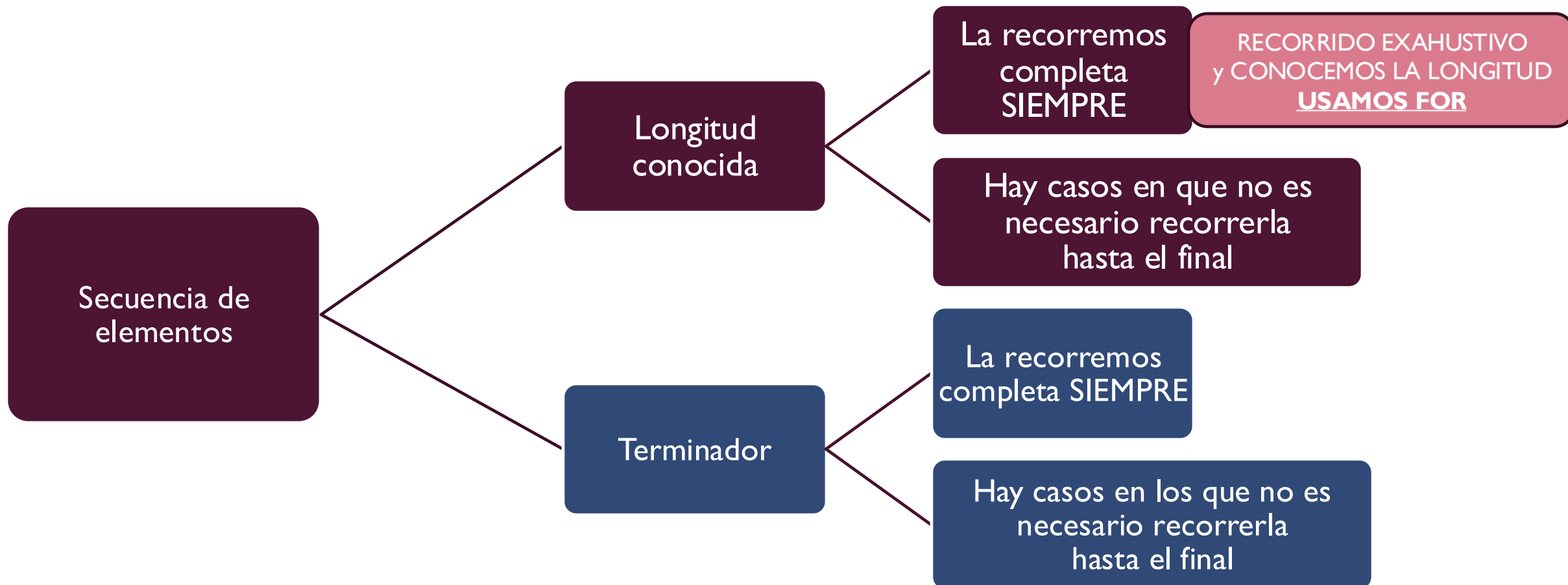
Si la secuencia es 82 12 45 74 36 con anterioridad sabemos que es una secuencia que tiene 5 elementos.

- Hay un “terminador” que nos indica que finalizó la secuencia (un valor especial q no debe ser tratado como parte de la misma)

Si la secuencia es 82 12 45 74 36 se agregará un valor al final de la misma para indicar que finalizó. Este valor no debe ser un posible valor de la secuencia pero debe ser del mismo tipo que el resto de los elementos. Por ejemplo si sabemos que todos los números son positivos, un posible terminador sería el -1.

De esta forma el ingreso de la secuencia sería: 82 12 45 74 36 -1

ESCENARIOS POSIBLES



CASO 1a: LONGITUD CONOCIDA

RECORRIDO EXAHUSTIVO

Problema 1. Se desea recorrer una secuencia de números positivos y contar cuántos elementos pares hay en la misma. Asumimos que la **longitud** de la secuencia es conocida antes del ingreso de la secuencia.

LONGITUD: 7 elementos
Secuencia 10 3 221 55 14 1035 822

Respuesta: hay 3 elementos pares

LONGITUD: 2 elementos
Secuencia 107 133

Respuesta: hay 0 elementos pares

LONGITUD: 0 elementos
Secuencia -

Respuesta: hay 0 elementos pares

ALGORITMO

Algoritmo ContarEnSecuencia

DE: LONGITUD (entero), secuencia de elementos

DS: cantidadPares (entero)

Daux: elem

Leer(LONGITUD) // Pedir al usuario la longitud (cant de elementos) de la secuencia que va a ingresar luego

CantidadPares $\leftarrow$ 0

Mientras (LONGITUD > 0)

hacer

Leer(elem)

LONGITUD $\leftarrow$ LONGITUD - 1 }

// Si el elemento que leí es par, lo cuento

Si (elem MOD 2 = 0)

entonces

CantidadPares $\leftarrow$ CantidadPares + 1

Cada vez que leemos un elemento, la longitud de la secuencia se decrementa en uno. Tenemos una secuencia con un elemento menos que antes.

PROGRAMA

```
Program ContarEnSecuencia;  
Var longitud, elem, cantidadPares: integer;  
Begin  
  writeln('INGRESE LA CANTIDAD DE ELEMENTOS de la secuencia: ');  
  readln(longitud);  
  
  writeln('Ingrese los elementos de la secuencia separados por un espacio y finalice con enter.');
```

cantidadPares:=0;

```
  while (longitud > 0) do  
  begin  
    read(elem); {cuando entro al while es porq la long es mayor que cero y esto indica que hay al menos un elemento para leer}  
    longitud:=longitud - 1; {como leí un elemento, ahora queda una elemento menos en la secuencia}  
    if (elem mod 2 = 0) {verifico si el elemento que leí es par}  
    then  
      cantidadPares:=cantidadPares+1; {si se cumple que es par, incremento la cantidad de numeros pares encontrados}  
    end;  
  writeln('La cantidad de elementos pares que se encontraron en la secuencia es: ', cantidadPares);  
  
  readln; {este readln TOMA EL ENTER que se ingresó para finalizar la secuencia}  
  readln; {este readln es el que queda esperando a que el usuario vea la consola y luego presione enter}  
end.
```

PROGRAMA VERSION MEJORADA

Problema I: RECORRIDO EXAHUSTIVO
y CONOCEMOS LA LONGITUD
USAMOS FOR

```
Program ContarEnSecuencia;  
Var longitud,i, elem, cantidadPares: integer;  
Begin  
writeln('INGRESE LA CANTIDAD DE ELEMENTOS de la secuencia: ');  
readln(longitud);  
  
writeln('Ingrese los elementos de la secuencia separados por un espacio y finalice con enter.');
```

cantidadPares:=0;
for i:=1 to longitud do
begin
 read(elem); {cuando entro al while es porq la long es mayor que cero y esto indica que hay al menos un elemento para leer}
 if (elem mod 2 = 0) {verifico si el elemento que leí es par}
 then
 cantidadPares:=cantidadPares+1; {si se cumple que es par, incremento la cantidad de numeros pares encontrados}
end;
writeln('La cantidad de elementos pares que se encontraron en la secuencia es: ', cantidadPares);

readln; {este readln TOMA EL ENTER que se ingresó para finalizar la secuencia}
readln; {este readln es el que queda esperando a que el usuario vea la consola y luego presione enter}
end.

CASO 1b: LONGITUD CONOCIDA

RECORRIDO **NO** EXAHUSTIVO

Problema 2. Se desea recorrer una secuencia de números positivos y determinar si hay algún elemento par en la misma. Asumimos que la longitud de la secuencia es conocida.

LONGITUD: 7 elementos

Secuencia 10 3 221 55 14 1035 822

Respuesta: HAY UNO!

LONGITUD: 2 elementos

Secuencia 107 133

Respuesta: NO HAY NINGUNO

LONGITUD: 0 elementos

Secuencia -

Respuesta: NO HAY NINGUNO

ALGORITMO

Algoritmo HayEnSecuencia

DE: LONGITUD (entero), secuencia de elementos

DS: HayPar (lógico)

Daux: elem

Leer(LONGITUD) // Pedir al usuario la longitud (cant de elementos) de la secuencia que va a ingresar luego

HayPar $\leftarrow$ FALSO

Mientras ((LONGITUD > 0) Y (HayPar=FALSO))

hacer

 Leer(elem)

 LONGITUD $\leftarrow$ LONGITUD - 1

 Si (elem MOD 2 = 0)

 entonces

 HayPar $\leftarrow$ VERDADERO // encuentro un elemento par y CAMBIO el valor del dato de salida

Dos casos (CONDICIONES) para salir del bucle:

- Si se termina la secuencia (LONGITUD es cero) por lo tanto no encuentre ningún par
- Si encuentro un par (HayPar es Verdadero) y no necesito seguir buscando

PROGRAMA

Problema 2: NO PODEMOS USAR FOR!! Porque si encuentro un numero par quiero dejar de recorrer y el FOR no se puede cortar antes de terminar de recorrer el intervalo especificado.

```
Program HayEnSecuencia;  
Var longitud, elem, cantidadPares: integer;  
    hayPar: boolean;  
Begin  
    writeln('INGRESE LA CANTIDAD DE ELEMENTOS de la secuencia: ');  
    readln(longitud);  
  
    writeln('Ingrese los elementos de la secuencia separados por un espacio y finalice con enter.');
```

HayPar:=False;

```
while ((longitud > 0) AND (HayPar=FALSE)) do  
begin  
    read(elem);  
    longitud:=longitud-1;  
    if (elem mod 2 = 0)    {verifico si el elemento que leí es par}  
    then  
        HayPar:=TRUE; {si se cumple que es par, cambio el valor de la variable lógica y con esto se cortará el while}  
end;  
  
if (HayPar)  
then writeln('HAY AL MENOS UN ELEMENTO PAR EN LA SECUENCIA')  
else  writeln('NO HAY NINGUN ELEMENTO PAR EN LA SECUENCIA');
```

readln; {este readln TOMA EL ENTER que se ingresó para finalizar la secuencia}
readln; {este readln es el que queda esperando a que el usuario vea la consola y luego presione enter}
end.

CASO 1b: LONGITUD CONOCIDA

RECORRIDO **NO** EXAHUSTIVO

Problema 3. Se desea recorrer una secuencia de números positivos y determinar si hay exactamente 3 elementos pares hay en la misma. Asumimos que la longitud de la secuencia es conocida.

Pensar al menos 5 ejemplos interesantes de TESTEO
para el algoritmo.

CASO 2a: LONGITUD DESCONOCIDA - TERMINADOR

RECORRIDO EXAHUSTIVO

Problema 4. Se desea recorrer una secuencia de números positivos y contar cuántos elementos pares hay en la misma. Asumimos que la secuencia finaliza con un elemento especial (terminador, -1).

Secuencia 10 3 221 55 14 1035 -1

Respuesta: hay 2 elementos pares

Secuencia 107 133 -1

Respuesta: hay 0 elementos pares

Secuencia -1

Respuesta: hay 0 elementos pares

ALGORITMO

Algoritmo cuantosEnSecuenciaCASO2

DE: secuencia de elementos

DS: CantidadPares

Daux: elem

CantidadPares $\leftarrow$ 0

REPETIR mientras no es fin de la secuencia

 leer elemento

 si elemento es terminador, fin de la secuencia es verdadero

 sino

 si elemento es par, incrementamos la CantidadPares

ALGORITMO REFINADO

Algoritmo cuantosEnSecuenciaCASO2

DE: secuencia de elementos

DS: CantidadPares

Daux: elem (entero), finSEC (lógico)

CantidadPares $\leftarrow$ 0

finSEC $\leftarrow$ FALSO

Mientras (finSEC=FALSO) // finSEC se pondrá en VERDADERO cuando se lea el terminador
hacer

 leer(elem)

 si elem=-1

 entonces finSEC $\leftarrow$ VERDADERO

 sino

 si (elem mod 2 = 0) entonces CantidadPares $\leftarrow$ CantidadPares + 1

ALGORITMO REFINADO

Algoritmo cuantosEnSecuenciaCASO2

DE: secuencia de elementos

DS: CantidadPares

Daux: elem (entero), finSEC (lógico)

CantidadPares $\leftarrow$ 0

finSEC $\leftarrow$ FALSO

Mientras (finSEC=FALSO) // finSEC se pondrá en VERDADERO cuando se lea el terminador
hacer

 leer(elem)

 si elem=-1

 entonces finSEC $\leftarrow$ VERDADERO

sino // solo verifico si es par en caso de que no sea el terminador. El terminador no puede ser analizado
 si (elem mod 2 = 0) entonces CantidadPares $\leftarrow$ CantidadPares + 1

PROGRAMA

```
Program cuantosEnSecuenciaCaso2;  
Var elem, cantidadPares: integer;  
    finSEC: boolean;  
const terminador=-1;
```

Begin

```
writeln('Ingrese los elementos de la secuencia separados por un espacio y finalice con un -1.');
```

finSEC:=False;
cantidadPares:=0;
while (finSEC=FALSE) do

begin

```
    read(elem);  
    if (elem=terminador)  
    then  
        finSEC:=TRUE  
    else  
        if (elem mod 2 = 0)    {verifico si el elemento que leí es par}  
        then  
            cantidadPares:=cantidadPares+1; {si se cumple que es par, cambio el valor de la variable lógica y con esto se cortará el while}  
        end;  
    end;
```

```
writeln('En la secuencia habia ', cantidadPares, ' elementos pares.');
```

```
readln; {este readln TOMA EL ENTER que se ingresó al finalizar la secuencia}  
readln; {este readln es el que queda esperando a que el usuario vea la consola y luego presione enter}  
end.
```

CASO 2b: LONGITUD DESCONOCIDA - TERMINADOR

RECORRIDO **NO** EXAHUSTIVO

Problema 5. Se desea recorrer una secuencia de números positivos y determinar si hay algún elemento par en la misma. Asumimos que la secuencia finaliza con un terminador.

Secuencia 10 3 221 55 14 1035 -1

Respuesta: HAY UNO!

LONGITUD: 2 elementos
Secuencia 107 133

Respuesta: NO HAY NINGUNO

LONGITUD: 0 elementos
Secuencia -

Respuesta: NO HAY NINGUNO

ALGORITMO REFINADO

Algoritmo hayEnSecuenciaCASO2

DE: secuencia de elementos

DS: hayPar (lógico)

Daux: elem (entero), finSEC (lógico)

hayPar $\leftarrow$ FALSO

finSEC $\leftarrow$ FALSO

Mientras ((finSEC=FALSO) y (hayPar=FALSO))

hacer

 leer(elem)

 si elem=-1

 entonces finSEC $\leftarrow$ VERDADERO

 sino

 si (elem mod 2 = 0) entonces hayPar $\leftarrow$ VERDADERO

ALGORITMO MAS REFINADO

Algoritmo hayEnSecuenciaCASO2

DE: secuencia de elementos

DS: hayPar (lógico)

Daux: elem (entero)

repetir

 leer(elem)

Hasta (elem = -1) o (elem mod 2 = 0)

hayPar $\leftarrow$ elem mod 2 = 0

ESPERO QUE LES HAYA SERVIDO ESTE REPASO CON MAS EJEMPLOS!

- Hasta la próxima!

